

ZeRO-2: A Concrete Walkthrough with Actual Numbers

1. Our Tiny Transformer Setup (Same as ZeRO-1)

Hidden dimension (d)

= 4

Number of attention heads

= 2 → head dimension $d_k = 4/2 = 2$

FFN inner dimension

= 16 (4× expansion)

Vocab size

= 8

Sequence length (T)

= 3 (3 tokens)

Number of GPUs

= 2 (GPU-0 and GPU-1)

2. Listing Every Parameter (with shapes)

Following the architecture diagram (LN1 → Attention → residual → LN2 → FFN → residual → W_vocab → softmax):

Layer	Name	Shape	#Elements
LayerNorm 1	γ_1	(4,)	4
	β_1	(4,)	4
Attention:			
Query weights	W_q	(4, 4)	16
Key weights	W_k	(4, 4)	16
Value weights	W_v	(4, 4)	16
Output proj	W_o	(4, 4)	16
LayerNorm 2	γ_2	(4,)	4
	β_2	(4,)	4
FFN:			
Up-projection	W_1	(4, 16)	64
Bias 1	b_1	(16,)	16
Down-projection	W_2	(16, 4)	64
Bias 2	b_2	(4,)	4
Output head	W_{vocab}	(4, 8)	32
TOTAL PARAMETERS:			260

3. Concrete Parameter Values (Initialised)

Same initial weights as our ZeRO-1 walkthrough (identical starting model).

W_q (Query weights) — shape (4, 4):

```
W_q = [ 0.12  0.34 -0.21  0.05 ]  
       [-0.15  0.22  0.11 -0.08 ]  
       [ 0.30 -0.10  0.18  0.27 ]  
       [-0.05  0.14 -0.33  0.09 ]
```

W_k (Key weights) — shape (4, 4):

```
W_k = [ 0.20 -0.11  0.07  0.15 ]  
       [ 0.03  0.28 -0.14  0.10 ]  
       [-0.22  0.06  0.31 -0.05 ]  
       [ 0.17 -0.09  0.02  0.24 ]
```

W_v (Value weights) — shape (4, 4):

```
W_v = [ 0.08  0.19 -0.12  0.25 ]  
       [-0.07  0.33  0.04 -0.16 ]  
       [ 0.21 -0.03  0.15  0.11 ]  
       [-0.14  0.08  0.26 -0.02 ]
```

W₁ (FFN up-projection) — shape (4, 16): (showing first 4 of 16 columns)

```
W_1 = [ 0.10  0.05 -0.22  0.13  ... ]   (4 rows × 16 cols = 64 values)  
       [-0.08  0.17  0.03 -0.11  ... ]  
       [ 0.25 -0.06  0.14  0.09  ... ]  
       [-0.03  0.21 -0.07  0.18  ... ]
```

W_{vocab} (output head) — shape (4, 8):

```
W_vocab = [ 0.11 -0.05  0.18  0.03 -0.14  0.22 -0.09  0.07 ]  
          [-0.06  0.13  0.02 -0.17  0.08  0.04  0.20 -0.11 ]  
          [ 0.15 -0.02  0.09  0.21 -0.08  0.12 -0.03  0.16 ]  
          [-0.10  0.07 -0.13  0.05  0.19 -0.06  0.14  0.01 ]
```

LayerNorm parameters:

```
γ1 = [1.0, 1.0, 1.0, 1.0]   (initialized to ones)  
β1 = [0.0, 0.0, 0.0, 0.0]   (initialized to zeros)  
γ2 = [1.0, 1.0, 1.0, 1.0]  
β2 = [0.0, 0.0, 0.0, 0.0]
```

4. Memory Accounting: ZeRO-2 vs ZeRO-1 vs Baseline

The key insight of ZeRO-2: **there is no reason to keep the full gradient vector on every GPU**. Each GPU only needs the averaged gradient for its own optimizer slice. So we partition gradients too.

What	Precision	Bytes/element
Parameter (for fwd/bwd)	BF16	2
Gradient	BF16	2
Optimizer states (Adam)		
Master copy of parameter	FP32	4
First moment m	FP32	4
Second moment v	FP32	4

Comparison (260 parameters, 2 GPUs):

	No ZeRO (per GPU)	ZeRO-1 (per GPU)	ZeRO-2 (per GPU)
Params (BF16, full):	260×2= 520	260×2= 520	260×2= 520
Grads (BF16):	260×2= 520	260×2= 520	130×2= 260 ← PARTITIONED!
Optimizer (FP32):	260×12=3120	130×12=1560	130×12=1560
TOTAL per GPU:	4,160	2,600	2,340
SAVING vs baseline:	—	37.5%	43.8%

The extra saving over ZeRO-1: **260 bytes** (we dropped half the gradient storage per GPU).

5. The Partition: Who Owns What

Same partition as ZeRO-1 — we flatten all 260 parameters into a 1D vector and split:

Flat parameter vector (260 elements):	
[$\gamma_1(4)$ $\beta_1(4)$ $W_q(16)$ $W_k(16)$ $W_v(16)$ $W_o(16)$ $\gamma_2(4)$ $\beta_2(4)$ $W_1(64)$ $b_1(16)$ $W_2(64)$ $b_2(4)$ $W_{vocab}(32)$]	
indices 0..129 → GPU-0 owns indices 130..259 → GPU-1 owns	

But now "owns" means MORE than in ZeRO-1:

GPU-0 is responsible for slice [0:130]:

- ✓ Optimizer states: $m_0[130], v_0[130], p_0[130]$ in FP32 ← same as ZeRO-1
- ✓ Averaged gradients: $avg_g[0:130]$ in BF16 ← NEW in ZeRO-2!

GPU-1 is responsible for slice [130:260]:

- ✓ Optimizer states: $m_1[130], v_1[130], p_1[130]$ in FP32
- ✓ Averaged gradients: $avg_g[130:260]$ in BF16 ← NEW in ZeRO-2!

What BOTH GPUs still store (replicated):

Full parameters $\theta[260]$ in BF16 (needed for forward & backward pass)
That's it! Nothing else is fully replicated.

6. Walking Through One Training Step

Input

Same as ZeRO-1 walkthrough — our 3-token input sequence:

$x_0 = [0.5, -0.3, 0.8, 0.1]$ ← token 0
[0.2, 0.7, -0.1, 0.4] ← token 1
[-0.6, 0.3, 0.5, -0.2] ← token 2

GPU-0 gets micro-batch A, GPU-1 gets micro-batch B.

STEP 1 + 2 (FUSED): Forward & Backward Pass WITH Reduce-Scatter

This is the critical difference from ZeRO-1. In ZeRO-1, we did:

1. Full backward → store all 260 local gradients → then reduce-scatter

In ZeRO-2, we **fuse** reduce-scatter into the backward pass itself. Gradients are reduce-scattered **as soon as they are computed**, and the local full gradient is **immediately discarded**.

Forward Pass (identical to ZeRO-1):

Each GPU independently runs the full model:

```

 $x_1 = \text{LN1}(x_0) \rightarrow (3, 4)$ 
 $Q = x_1 \cdot W_q, K = x_1 \cdot W_k, V = x_1 \cdot W_v \rightarrow \text{each } (3, 4)$ 
 $x_{\text{attn}} = \text{Attention}(Q, K, V) \cdot W_o \rightarrow (3, 4)$ 
 $x_{\text{res1}} = x_0 + x_{\text{attn}} \rightarrow (3, 4)$ 
 $x_2 = \text{LN2}(x_{\text{res1}}) \rightarrow (3, 4)$ 
 $x_{\text{ffn}} = \text{ReLU}(x_2 \cdot W_1 + b_1) \cdot W_2 + b_2 \rightarrow (3, 4)$ 
 $x_{\text{final}} = x_{\text{res1}} + x_{\text{ffn}} \rightarrow (3, 4)$ 
 $\text{logits} = x_{\text{final}} \cdot W_{\text{vocab}} \rightarrow (3, 8)$ 
 $z = \text{softmax}(\text{logits}) \rightarrow (3, 8)$ 
 $\text{Loss} = \text{CrossEntropy}(z, \text{targets}) \rightarrow \text{scalar}$ 

```

Backward Pass — Layer by Layer with Immediate Reduce-Scatter:

Backprop goes in **reverse** order. Watch what happens to gradient memory:

Backprop through W_{vocab} (in GPU-1's slice [130:260]):

Both GPUs compute locally:

GPU-0: $g_{\text{vocab}}^{\text{(A)}} = \partial \text{Loss}_A / \partial W_{\text{vocab}} \rightarrow (4, 8) = 32 \text{ values}$

GPU-1: $g_{\text{vocab}}^{\text{(B)}} = \partial \text{Loss}_B / \partial W_{\text{vocab}} \rightarrow (4, 8) = 32 \text{ values}$

IMMEDIATELY reduce-scatter for this chunk:

→ GPU-1 receives: $\text{avg_g_vocab} = (g_{\text{vocab}}^{\text{(A)}} + g_{\text{vocab}}^{\text{(B)}}) / 2$

→ GPU-0 receives: nothing for W_{vocab} (not its slice!)

→ GPU-0 DISCARDS $g_{\text{vocab}}^{\text{(A)}}$ from memory right now

GPU-0 gradient memory at this point: 0 values stored

GPU-1 gradient memory at this point: 32 values (avg_g_vocab)

Backprop through W_2, b_2 (in GPU-1's slice):

Both GPUs compute $g_{W_2}^{\text{(local)}}, g_{b_2}^{\text{(local)}}$

Immediately reduce-scatter:

→ GPU-1 receives avg_g_W2 (64 values) and avg_g_b2 (4 values)

→ GPU-0 DISCARDS both immediately

GPU-0 gradient memory: 0 values

GPU-1 gradient memory: $32 + 64 + 4 = 100 \text{ values}$

Backprop through W_1, b_1 (split across both slices!):

W_1 has 64 elements. The first 2 rows (the portion in indices 0–129) belong to GPU-0's slice, and the remaining portion belongs to GPU-1's slice.

Both GPUs compute $g_{W_1}^{(local)}$ (64 values), $g_{b_1}^{(local)}$ (16 values)

Reduce-scatter:

- GPU-0 receives avg gradients for its portion of W_1 (~2 rows $\approx$ 32 values)
- GPU-1 receives avg gradients for its portion of $W_1 + b_1$
- Each GPU DISCARDS the rest

GPU-0 gradient memory: ~32 values (its slice of W_1)

GPU-1 gradient memory: $100 + \sim 48 = \sim 148$ values... but wait—

GPU-1 only KEEPS gradients for indices 130–259. It's accumulating exactly 130 values.

Backprop continues through $\gamma_2, \beta_2, W_o, W_v, W_k, W_q, \beta_1, \gamma_1$ (all in GPU-0's slice):

For each parameter, both GPUs compute local gradients:

GPU-0: $g_q^{(A)} = \begin{bmatrix} 0.023 & -0.011 & 0.045 & -0.008 \\ -0.031 & 0.019 & -0.007 & 0.014 \\ 0.012 & -0.028 & 0.033 & -0.005 \\ -0.016 & 0.009 & -0.021 & 0.038 \end{bmatrix}$

GPU-1: $g_q^{(B)} = \begin{bmatrix} 0.017 & -0.025 & 0.031 & -0.013 \\ -0.009 & 0.041 & -0.018 & 0.006 \\ 0.028 & -0.014 & 0.022 & -0.035 \\ -0.020 & 0.016 & -0.012 & 0.027 \end{bmatrix}$

Immediately reduce-scatter:

- GPU-0 receives: $avg_g_q = (g_q^{(A)} + g_q^{(B)}) / 2$

$= \begin{bmatrix} 0.020 & -0.018 & 0.038 & -0.0105 \\ -0.020 & 0.030 & -0.0125 & 0.010 \\ 0.020 & -0.021 & 0.0275 & -0.020 \\ -0.018 & 0.0125 & -0.0165 & 0.0325 \end{bmatrix}$

- GPU-1 receives: NOTHING for W_q (not its slice!)
- GPU-1 DISCARDS $g_q^{(B)}$ immediately

End of Backward Pass — What Each GPU Holds:

GPU-0: $avg_g[0:130]$ — exactly 130 averaged gradient values in BF16
NOTHING for indices 130–259 (all discarded during backward)

GPU-1: $avg_g[130:260]$ — exactly 130 averaged gradient values in BF16
NOTHING for indices 0–129 (all discarded during backward)

Compare to ZeRO-1, where BOTH GPUs held all 260 gradient values after backward, and THEN did reduce-scatter as a separate step.

Peak Gradient Memory Comparison (The Real Win):

ZeRO-1:

During backward, each GPU accumulates the FULL gradient: 260 values

After reduce-scatter, each GPU could free the non-local gradients

PEAK gradient memory: $260 \times 2 = 520$ bytes per GPU

ZeRO-2:

During backward, each GPU only KEEPS gradients for its own slice

Non-local gradients are reduce-scattered and discarded immediately

PEAK gradient memory: $130 \times 2 = 260$ bytes per GPU ← HALF!

In practice, it's slightly more nuanced — the GPU temporarily holds a layer's worth of local gradients before the reduce-scatter for that layer fires. But these are immediately freed, so peak memory is much lower than holding the full gradient tensor.

STEP 3: Optimizer Step (Update Local Slice)

Identical to ZeRO-1. Each GPU runs Adam only on its 130-element slice.

GPU-0 updates W_q (Adam step, iteration $t=1$):

$lr=0.001, \beta_1=0.9, \beta_2=0.999, \epsilon=1e-8$

For $W_q[0,0]$:

Current master param (FP32): $p = 0.12$

Averaged gradient: $g = 0.020$

$m_{\text{new}} = 0.9 \times 0.0 + 0.1 \times 0.020 = 0.002$

$v_{\text{new}} = 0.999 \times 0.0 + 0.001 \times 0.0004 = 0.0000004$

$\hat{m} = 0.002 / (1 - 0.9^1) = 0.02$

$\hat{v} = 0.0000004 / (1 - 0.999^1) = 0.0004$

$p_{\text{new}} = 0.12 - 0.001 \times 0.02 / (\sqrt{0.0004} + 1e-8)$

$= 0.12 - 0.001 \times 0.02 / 0.02$

$= 0.12 - 0.001$

$= 0.119$

Cast to BF16: $W_q[0,0] \approx 0.1191$

GPU-0 repeats for all 130 elements. GPU-1 does the same for its 130 elements.

```
GPU-0: new_params[0:130] in BF16 ✓ | params[130:260] still OLD ✗
GPU-1: new_params[130:260] in BF16 ✓ | params[0:130] still OLD ✗
```

Identical to ZeRO-1. Each GPU broadcasts its updated slice:

After all-gather, both GPUs hold:

```

θ[260] = [new_params[0:130] | new_params[130:260]] ← full updated model

```

```
W_q (updated) = [ 0.119  0.341 -0.211  0.051 ]
                  [-0.149  0.219  0.111 -0.081 ]
                  [ 0.299 -0.099  0.179  0.271 ]
                  [-0.049  0.139 -0.329  0.089 ]
```

STEP 5: End of Step — What Lives Where

GPU-0	
$\theta[260]$ in BF16 (FULL model, updated)	= 520 bytes
avg_g[0:130] in BF16 (can be freed now)	= 260 bytes
—— Optimizer (ONLY slice 0:130) ——	
$m_0[130]$ in FP32 (first moments)	= 520 bytes
$v_0[130]$ in FP32 (second moments)	= 520 bytes
$p_0[130]$ in FP32 (master params)	= 520 bytes
TOTAL: 2,340 bytes (was 2,600 in ZeRO-1)	
(was 4,160 without ZeRO)	
GPU-1	
$\theta[260]$ in BF16 (FULL model, updated)	= 520 bytes
avg_g[130:260] in BF16 (can be freed now)	= 260 bytes

——— Optimizer (ONLY slice 130:260) ———		
m ₁ [130] in FP32 (first moments)	= 520 bytes	
v ₁ [130] in FP32 (second moments)	= 520 bytes	
p ₁ [130] in FP32 (master params)	= 520 bytes	
TOTAL: 2,340 bytes		

7. Communication Cost Analysis

ZeRO-1:

Reduce-Scatter (after backward): $260 \times 2 = 520$ bytes

All-Gather (after optimizer): $260 \times 2 = 520$ bytes

TOTAL: 1,040 bytes

ZeRO-2:

Reduce-Scatter (fused into backward, chunk by chunk):

Total data scattered = $260 \times 2 = 520$ bytes (same total, just pipelined)

All-Gather (after optimizer): $260 \times 2 = 520$ bytes

TOTAL: 1,040 bytes

Standard All-Reduce (no ZeRO):

TOTAL: 1,040 bytes

All three have IDENTICAL communication volume! The difference is purely about *when* communication happens and *how long* data is kept in memory. ZeRO-2 pipelines the reduce-scatter into the backward pass, allowing immediate gradient memory reclamation.

8. Side-by-Side: What Happens During Backward

This is where the difference really shows. Let's watch gradient memory frame-by-frame during backward:

ZeRO-1
(GPU-0 memory)

ZeRO-2
(GPU-0 memory)

Backprop W_{vocab} :

g_{vocab} computed → stored locally → reduce-scatter fires

Memory: +32 → 32 values → GPU-0 discards (not its slice)

Memory: 0 values

Backprop W_2 :

g_{W_2} computed → stored locally → reduce-scatter fires

Memory: +64 → 96 values → GPU-0 discards (not its slice)

Memory: 0 values

Backprop b_2 :

g_{b_2} computed → stored locally → reduce-scatter fires

Memory: +4 → 100 values → GPU-0 discards

Memory: 0 values

Backprop W_1 :

g_{W_1} computed → stored locally → reduce-scatter fires

Memory: +64 → 164 values → GPU-0 KEEPS its slice (~32 vals)

Memory: 32 values

Backprop b_1 :

g_{b_1} computed → stored locally → reduce-scatter fires

Memory: +16 → 180 values → GPU-0 discards

Memory: 32 values

... (continuing through $\gamma_2, \beta_2, W_o, W_v, W_k, W_q, \beta_1, \gamma_1$) ...

Backprop W_q :

Memory: +16 → 244 values → GPU-0 KEEPS avg_g_q (16 vals)

Memory: ~114 values

End of backward:

Memory: 260 values (FULL) 130 values (HALF)

↓

↓

Then reduce-scatter → 130 values

Already done! → 130 values

ZeRO-1 peak gradient memory: 260 values (520 bytes) ZeRO-2 peak gradient memory: ~130 values (260 bytes) (plus one layer's temporary buffer)

9. Scaling to a Real Model: 7B Parameters on 8 GPUs

	No ZeRO (per GPU)	ZeRO-1 (8 GPUs)	ZeRO-2 (8 GPUs)
Params (BF16):	14.0 GB	14.0 GB	14.0 GB
Gradients (BF16):	14.0 GB	14.0 GB	1.75 GB ← +8!
Optimizer:			
m (FP32):	28.0 GB	3.5 GB	3.5 GB
v (FP32):	28.0 GB	3.5 GB	3.5 GB
master p (FP32):	28.0 GB	3.5 GB	3.5 GB
TOTAL per GPU:	112.0 GB	38.5 GB	26.25 GB
Saving vs baseline:	—	65.6%	76.6%
Extra saving vs Z1:	—	—	12.25 GB
Extra comm. cost:	—	none	none

10. Why Not Always Use ZeRO-2 Over ZeRO-1?

As your notes say: **there is no real overhead to using ZeRO-2 over ZeRO-1 besides implementation complexity, and indeed ZeRO-2 is usually the better option.**

The communication volume is identical. The only differences are:

Dimension	ZeRO-1	ZeRO-2
Gradient memory	Full (N/N)	Partitioned (1/N)
Communication timing	Reduce-scatter AFTER full backward	Reduce-scatter DURING backward (pipelined)
Implementation	Simple (one RS call)	More complex (bucket-by-bucket RS fused with backward)
Communication volume	Identical	Identical
Optimizer step	Identical	Identical
All-gather	Identical	Identical

The fused reduce-scatter in ZeRO-2 is slightly more complex to implement (you need to hook into the autograd engine and fire reduce-scatter per-bucket as gradients become ready), but modern frameworks like DeepSpeed handle this transparently.

11. Summary: ZeRO-1 → ZeRO-2, The One Key Change

ZeRO-1:

Backward pass: Compute all gradients → store full $g[260]$

Reduce-scatter: Separate step after backward → each GPU keeps $g[130]$

Optimizer step: Adam on local 130 elements

All-gather: Broadcast updated params

Per-GPU formula: $2P + 2P + 12P/N$ bytes (params + grads + optimizer)

ZeRO-2:

Backward pass: Compute gradients AND reduce-scatter simultaneously

→ each GPU ONLY EVER stores $g[130]$

→ full local gradients are NEVER stored

Optimizer step: Adam on local 130 elements (identical)

All-gather: Broadcast updated params (identical)

Per-GPU formula: $2P + 2P/N + 12P/N$ bytes (params + grads/N + optimizer/N)

↑↑↑

This is the ONLY change from ZeRO-1

For our tiny model ($P=260$, $N=2$):

ZeRO-1: $520 + 520 + 1560 = 2,600$ bytes

ZeRO-2: $520 + 260 + 1560 = 2,340$ bytes (260 bytes saved = gradient partition)

For 7B on 8 GPUs:

ZeRO-1: $14 + 14 + 10.5 = 38.5$ GB

ZeRO-2: $14 + 1.75 + 10.5 = 26.25$ GB (12.25 GB saved per GPU, for free!)

The progression to ZeRO-3 is now natural: if we can partition gradients (ZeRO-2), why not partition the parameters themselves? That eliminates the remaining $2P$ term — but at the cost of extra all-gather communication during the forward pass.